

Übung 7

Aufgabe 1 – Datenabhängigkeiten und Pipeline-Konflikte (5+5+3+2 Punkte)

Betrachte das folgende RISC-V Programm (siehe Abbildung 1) und nimm an, dass es auf einem RISC-V Prozessor mit Pipelining (siehe Abbildung 2) ausgeführt werden soll.

- Gib alle Datenabhängigkeiten zwischen den Befehlen des Programms an (RAW, WAR und WAW).
- Finde alle auftretenden Pipeline-Konflikte (Daten- und Steuer-Konflikte). Gib jeweils die involvierten Befehle an und erkläre warum es zu einem Konflikt kommt.
- Behebe alle Pipeline-Konflikte durch Einfügen der minimalen Anzahl an NOP-Befehlen.
- Minimiere die Anzahl der NOP-Befehle durch Umordnen der Befehle (ohne die Semantik des Programms zu verändern).

```
.globl main
.data
nums: .word 30
.text
main:
i1    addi t1, zero, 12
i2    la t0, nums
i3    lw t0, 0(t0)
i4    sub t2, t0, t1
i5    blt t1, t2, end
i6    and t2, t2, t1
end:
i7    add t2, t2, t1
i8    addi t3, zero, 10
```

a)

RAW-Abhängigkeiten:

- $i_1 \rightarrow i_4$: t_1 sub liest t_1
- $i_1 \rightarrow i_5$: t_1 blt verwendet t_1 zum vergleichen
- $i_1 \rightarrow i_6$: t_1 and liest t_1
- $i_1 \rightarrow i_7$: t_1 add liest t_1
- $i_2 \rightarrow i_3$: t_0 lw verwendet Adresse von nums
- $i_3 \rightarrow i_4$: t_0 sub liest t_0
- $i_4 \rightarrow i_5$: t_2 blt verwendet t_2 zum vergleichen
- $i_4 \rightarrow i_6$: t_2 and liest t_2
- $i_4 \rightarrow i_7$: t_2 add liest t_2 , falls in Branch geht
- $i_6 \rightarrow i_7$: t_2 add liest t_2 , falls Branch übersprungen wird

WAR-Abhängigkeiten:

- $i_5 \rightarrow i_6$: t_2 blt liest t_2 , and schreibt in t_2
- $i_5 \rightarrow i_7$: t_2 blt liest t_2 , add schreibt in t_2

- $i_6 \rightarrow i_7$: t_2 and liest t_2 , odd schreibt in t_2

WAW-Abhängigkeiten

- $i_2 \rightarrow i_3$: t_0 zuerst schreibt $la\ int_0$, dann lv
- $i_4 \rightarrow i_6$: t_2 zuerst schreibt $sub\ int_2$, dann and
- $i_4 \rightarrow i_7$: t_2 zuerst schreibt $sub\ int_2$, dann add
- $i_6 \rightarrow i_7$: t_2 zuerst schreibt $and\ int_2$, dann add

b)

Datenkonflikte

- $i_2 \rightarrow i_3$: t_0 i_3 braucht t_0 als Adresse, ohne NOPs liest i_3 in Decode alten Wert von t_0
RAW-Datenkonflikt
- $i_3 \rightarrow i_4$: t_0 i_4 braucht geladenen Wert aus Speicher, ohne NOPs liest i_4 zu früh
RAW-Datenkonflikt
- $i_4 \rightarrow i_5$: t_2 i_5 braucht Ergebnis von i_4 , um Branch-Vergleich durchzuführen. Ohne NOPs liest i_5 alten Wert von t_2
RAW-Datenkonflikt
- $i_4 \rightarrow i_6$: t_2 falls Branch nicht genommen wird, braucht i_6 Ergebnis von i_4 , ohne NOPs wäre der Wert noch im Register File
RAW-Datenkonflikt
- $i_4 \rightarrow i_7$: t_2 falls Branch genommen wird, springt Prog. zu i_7 , braucht dann den Wert aus i_4
RAW-Datenkonflikt
- $i_6 \rightarrow i_7$: t_2 falls Branch nicht genommen wird, verwendet i_7 Ergebnis von i_6 , ohne NOPs liest i_7 alten Wert

Steuerkonflikte

i_5 blt t_1, t_2, end

Während i_5 noch nicht entschieden hat, ob gesprungen wird oder nicht, werden bereits nach folgende Instruktionen geladen.

c)

```

.global main
.data
nums: .word 30

.text
main:
i1    addi t1, zero, 12
i2    la t0, nums
      nop
      nop
      nop
i3    lw t0, 0(t0)
      nop
      nop
      nop
i4    sub t2, t0, t1
      nop
      nop
      nop
i5    blt t1, t2, end
      nop
      nop
i6    and t2, t2, t1
      nop
      nop
      nop
      end:
i7    add t2, t2, t1
i8    addi t3, zero, 10

14 NOPs

```

d)

```

.global main
.data
nums: .word 30

.text
main:
i2    la t0, nums
i7    addi t1, zero, t2
i8    addi t3, zero, 10
      nop
i7    lw t0, 0(t0)
      nop
      nop
      nop
i4    sub t2, t0, t1
      nop
      nop
      nop
i5    blt t1, t2, end
      nop
      nop
i6    and t2, t2, t1
      nop
      nop
      nop
      end:
i7    add t2, t2, t1

12 NOPs

```